PROJET WIKICOMPTINES

Rapport

SANTHALINGAM Mathura
mathura.santhalingam@edu.univ-paris13.fr

INTRODUCTION

Le but du projet Wikicomptines étant de permettre à des utilisateurs de partager des comptines par le biais de leur terminal, il est primordial de les faire échanger à travers un processus client et un processus serveur.

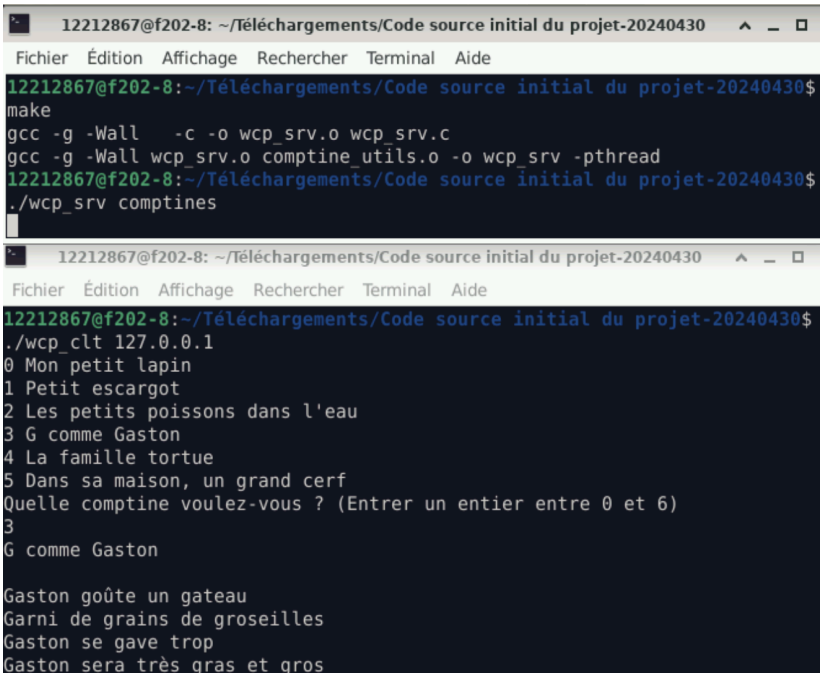
Résultat attendu :

```
$ ./wcp_clt 127.0.0.1
     0 Les petits poissons dans l'eau
     1 Petit escargot
     2 La famille tortue
     3 Mon petit lapin
     4 Dans sa maison, un grand cerf
     5 G comme Gaston
Quelle comptine voulez-vous ? (Entrer un entier entre 1 et 6) : 3
Mon petit lapin

Mon petit lapin
A bien du chagrin
Il ne saute plus
Il ne danse plus

Saute, saute, saute
Mon petit lapin
Saute, saute, saute
Tu auras du thym
```

Résultat obtenu :



DÉROULEMENT - COMPTINE_UTILS

Pour cela, avant toute chose, on fait appel à certaines fonctions que l'on retrouve dans le fichier **comptine_utils.c** et qui nous serviront tout au long du projet, aussi bien dans le processus serveur que dans le client. Il contient notamment les fonctions suivantes :

1. **int est_nom_fichier_comptine(char *nom_fich);**
   La fonction permet de nous assurer que les fichiers considérés sont bien au format ".cpt".

2. **read_until_nl(int fd, char *buf)**
   La fonction lit un caractère à la fois et les ajoute au buffer jusqu'à ce qu'un saut de ligne soit trouvé. Difficultés : ajout de saut de ligne et fin de chaîne de caractères au buffer.

3. **struct comptine *init_cpt_depuis_fichier(const char *dir_name, const char *base_name)**

La fonction se découpe en différentes étapes : en commençant par l'allocation dynamique pour une nouvelle structure comptine ainsi que pour ses différents champs puis elle y copie le nom du fichier en vérifiant bien qu'il s'agit d'une comptine. Pour ce qui est du nom du titre, pour y accéder, on est obligé d'ouvrir le fichier et y lire la première ligne et la copier grâce à la fonction **read_until_nl** et remplit le champ correspondant dans notre nouvelle structure.

4. **struct catalogue *creer_catalogue(const char *dir_name)**

Cette fonction ouvre le répertoire spécifié, initialise le catalogue puis compte le nombre de fichiers de comptines dans le répertoire et alloue de la mémoire pour le tableau de comptines du catalogue. Ensuite elle initialise chaque comptine depuis les fichiers dans le répertoire avant de renvoyer le catalogue.

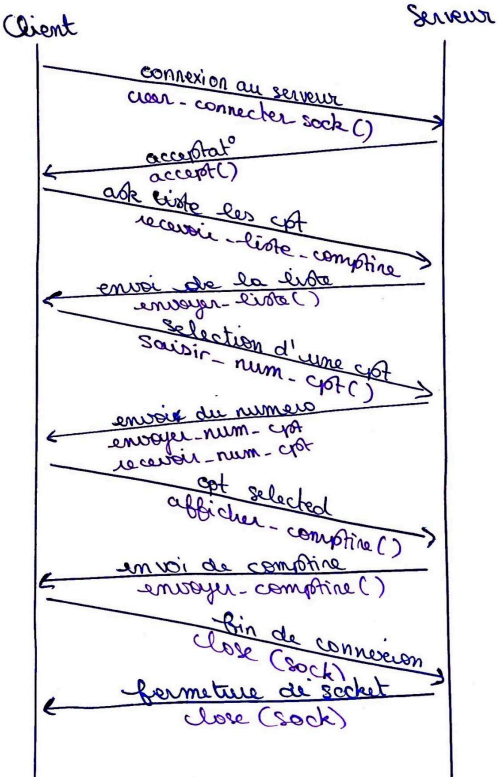
5. Il ne faut bien sûr pas oublier de **libérer la mémoire de la comptine et du catalogue**

Difficultés rencontrées dans cette étape : Ma fonction **est_nom_fichier_comptine** utilisait des pointeurs pour comparer les dernières lettres à ".cpt". Mais puisque l'on peut sans passer en comparant directement les cases, je ne savais pas quelle était la meilleure méthode puisque les deux marchent. J'ai finalement opté pour la seconde pour une question de simplicité et de lisibilité.

Dans la fonction struct catalogue ***creer_catalogue(const char *dir_name)**, une fois le nombre de comptine su et qu'on se soit à nouveau placé au début par rewinddir, afin d'ouvrir et lire le fichier plus tard, j'avais d'abord utilisé strcat pour concaténer le chemin complet vers chaque fichier de comptine mais ça ne fonctionnait pas car il fallait d'abord initialisé la chaîne de caractères soit au moins initialisée au vide.

DÉROULEMENT : WCP_SRV ET WCP_CLT

Schéma explicatif des étapes
de communication entre le serveur et le client.



CÔTÉ SERVEUR

1. **creer_configurer_sock_ecoute()**
   Cette fonction crée une socket d'écoute attachée à un port et à toutes les adresses locales, puis la met en mode écoute via bind et listen.

2. **envoyer_liste()**
   Elle envoie la liste des comptines au client, chaque ligne contient un indice et le titre d'une comptine.

3. **recevoir_num_comptine()**
   Elle lit un entier sur 2 octets (numéro de la comptine) envoyé par le client et le convertit en ordre d'octets de la machine. On utilise **ntohs()** pour convertir l'ordre d'octets au niveau du réseau en ordre d'octets au niveau de l'hôte.

4. ****envoyer_comptine()**
   Cette fonction envoie le contenu de la comptine spécifiée au client.

5. ***fonction()**
   Cette fonction gère les échanges avec un client individuel (thread)..

Difficultés rencontrées dans cette étape : Même si la fonction **setsockopt** est optionnelle, avant que je la mette, un message d'erreur s'affichait bind: Address already in use. Il fallait également faire attention lors de la création correcte des threads pour chaque client, en m'assurant que la structure répondra à tous les besoins..

CÔTÉ CLIENT

1. ***creer_connecter_sock()**
   Elle crée une socket et se connecte au serveur à l'adresse IPv4 spécifiée et au port donné. J'ai utilisé inet_pton() pour la conversion d'une adresse IPv4.

2. **recevoir_liste_comptines()**
   Cette fonction lit et affiche la liste des comptines envoyée par le serveur.

3. **saisir_num_comptine()**
   Celle-ci demande à l'utilisateur de sélectionner une comptine parmi celles disponibles. En s'assurant que l'entrée est un entier valide et qu'elle se situe dans les limites spécifiées.

4. **envoyer_num_comptine()**
   Envoie le numéro de la comptine sélectionnée au serveur en ordre d'octets réseau par cette fois **htons()** pour convertir l'ordre d'octets hôte en ordre d'octets réseau.

5. **afficher_comptine()**
   Affiche le contenu de la comptine reçue du serveur.

Difficultés rencontrées lors de cette étape : création de la structure pour les thread, je ne savais pas s'il fallait y ajouter les deux champs ou s'il fallait ajouter un champ pour argv aussi.

RÉFÉRENCES :

Pour mieux visualiser le concept des sockets (avant de relire les CM) :

- https://www.youtube.com/watch?v=_lQ-3S4fJ0U&list=PLPyaR5G9aNDvs6TtdpLcVO43_jvxp4emI
- https://du-isn.gricad-pages.univ-grenoble-alpes.fr/2-sr/Reseaux/3--cours_reseaux--sockets_java.pdf

Les CM

REMERCIEMENTS :

EL OTMANI Youssef
JENY JEYARAJ John